FIAP

CHALLENGE CTMSP E MARINHA DO BRASIL

2TDSPS-SERGIO E AMIGOS

Sérgio Henrique S. Santos — RM 567254

Paulo Cavalcante Caroba — RM 566667

Renan da Silva Paulino — RM 566610

Turma: 2TDSPS

AMAZUL

API JAVA EM DOCKER NO MICROSOFT AZURE

Sprint 1

DevOps Tools & Cloud Computing

2026

SUMÁRIO

1 SOLUÇÃO E ARQUITETURA 3
1.1 Docker e Cloud Computing na solução 3
2 DOCKER E EXECUÇÃO NO AZURE 4
2.1 Criação e implantação 4
2.2 Acesso por SSH e Swagger 4
3 EXCLUSÃO E LINKS DA ENTREGA 5
3.1 Encerramento do ambiente 5
3.2 Repositórios e vídeo 5
3.3 Arquivos de entrega 5

1 SOLUÇÃO E ARQUITETURA

O projeto AMAZUL oferece uma API educativa Java Spring Boot com artigos, fontes e locais relacionados às aplicações da tecnologia nuclear. Nesta entrega, a API é empacotada em Docker e executada em uma máquina virtual Ubuntu no Microsoft Azure.

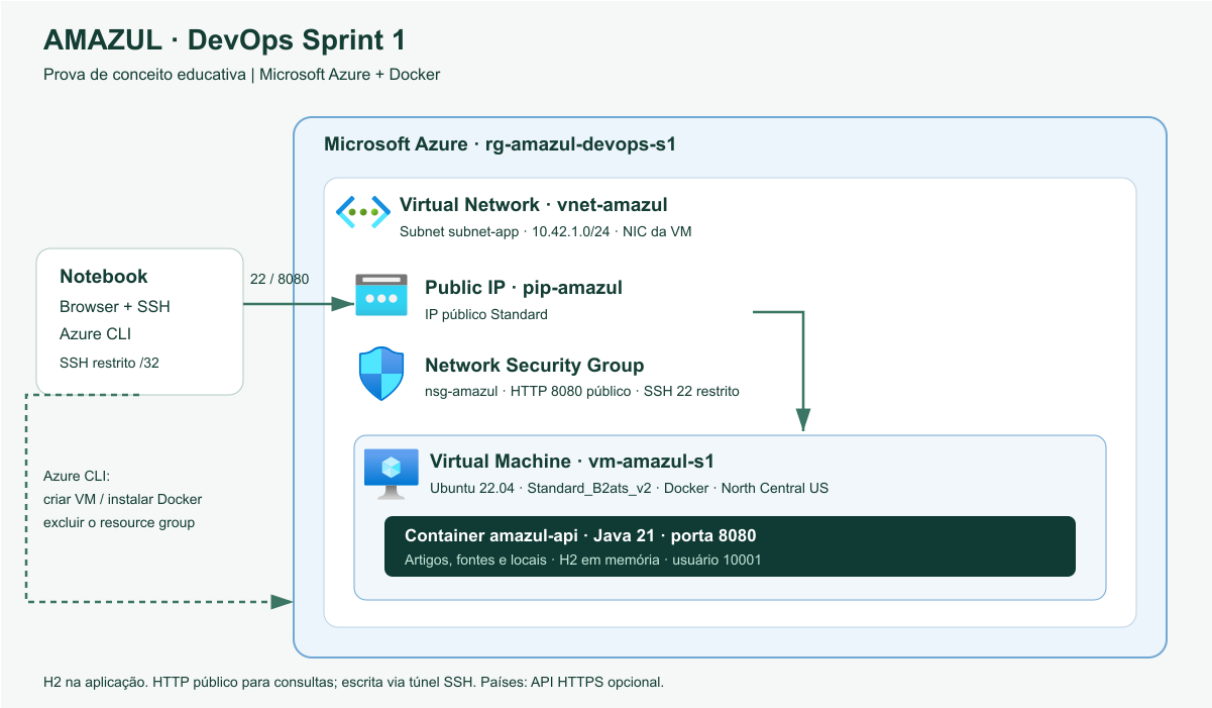


Figura 1 — Arquitetura da aplicação no Azure. Fonte: elaboração da equipe.

Ambiente: Ubuntu 22.04, VM Standard_B2ats_v2, região North Central US e Java 21. O grupo rg-amazul-devops-s1 reúne a VM, a rede virtual, a sub-rede, o NSG e o IP público. O banco H2 fica em memória; os dados são temporários. Oracle não faz parte deste ambiente.

1.1 Docker e Cloud Computing na solução

Docker empacota a API e suas dependências em uma imagem reproduzível. Azure disponibiliza a VM para executar o container. Azure CLI automatiza a criação e a exclusão dos recursos, enquanto SSH e SCP permitem instalar a aplicação e enviar os arquivos à VM.

2 DOCKER E EXECUÇÃO NO AZURE

O Dockerfile usa duas etapas: compila o projeto com Java 21 JDK e Maven e executa o JAR em Java 21 JRE. O container amazul-api usa o usuário 10001, limite de 640 MiB e healthcheck no endpoint /actuator/health. A porta da API é 8080.

2.1 Criação e implantação

No WSL, usar a pasta do projeto que contém o Dockerfile e scripts/. São necessários Azure CLI, Git, Python 3, SSH, SCP, tar e curl. O script conduz o login quando necessário e prepara a configuração, o código Java e as credenciais de escrita.

```
bash scripts/azure.sh create
bash scripts/azure.sh deploy
```

create provisiona os recursos e libera o SSH somente para o IPv4 atual do operador. deploy instala o Docker pela Azure CLI, envia o backend por SSH/SCP, constrói a imagem na VM e inicia o container. Após uma troca de rede, repetir create atualiza a origem permitida no SSH.

2.2 Acesso por SSH e Swagger

```
ssh -i ~/.ssh/amazul-s1 azureuser@IP_DA_VM
sudo docker ps
curl http://localhost:8080/actuator/health
```

Abrir http://IP_DA_VM:8080/swagger-ui/index.html no navegador. Consultas e Swagger são públicos na porta 8080; alterações exigem autenticação. Para cadastrar pelo Swagger, usar o túnel SSH informado pelo deploy e acessar http://localhost:18080/swagger-ui/index.html. As credenciais ficam em .local/api.env e não devem aparecer no vídeo ou no pacote.

A integração Country State City é opcional e exige uma chave própria. A API principal funciona sem essa chave.

3 EXCLUSÃO E LINKS DA ENTREGA

3.1 Encerramento do ambiente

Depois de gravar a aplicação e a VM no portal Azure, excluir o grupo de recursos e aguardar a confirmação de término:

```
CONFIRM_DESTROY=rg-amazul-devops-s1 bash scripts/azure.sh destroy
```

3.2 Repositórios e vídeo

Repositório DevOps

https://github.com/CHALLENGE-AMAZUL/DevOpsTools-CloudComputing

Backend Java

https://github.com/CHALLENGE-AMAZUL/backend-springboot

Vídeo demonstrativo no YouTube

https://youtu.be/BQV1CYI_7b8

O vídeo apresenta os scripts, a conexão SSH, o container Docker, o Swagger, a VM no portal Azure e o destroy, com legendas explicando as etapas.

3.3 Arquivos de entrega

O repositório reúne README, Dockerfile, scripts Azure CLI, arquitetura e instruções de execução.